LAPORAN UAS

ANALISIS PENGARUH METODE *PREPROCESSING OTSU*, *CANNY*, DAN KOMBINASI *OTSU–CANNY* TERHADAP KINERJA KLASIFIKASI CITRA SEL DARAH MERAH MENGGUNAKAN *EFFICIENTNETV2*

Nama Anggota:

Sisilia Leonita Pasaribu (187221056)
Alya Gita Ramadhani (187221059)
Octavia Putri Yudhistira (187221063)
Evitadewi Nur Hakimah (187221070)
Sara Nadya Maharani (187221092)

Mata Kuliah Computer Vision

S1 – Sistem Informasi

Fakultas Sains dan Teknologi

Universitas Airlangga

2025

DAFTAR ISI

DAFTAR ISI..... 1

DAFTAR GAMBAR.....2

DAFTAR TABEL.....3

BAB I DATA..... 1

BAB II *PREPROCESSING*.....3

2.1 *Import Libraries*..... 3

2.2 Pengaturan *Path Dataset*.....3

2.3 Pengaturan *Path Output*..... 4

2.4 Subfolder untuk Setiap Metode Pemrosesan.....4

2.5 Pembuatan Folder Otomatis..... 5

2.6 Fungsi untuk Memproses Gambar..... 6

2.7 Fungsi untuk Menyimpan Hasil Pemrosesan..... 7

2.8 Pemrosesan Semua Gambar dalam *Dataset*.....8

2.9 Output Pemrosesan Semua Gambar dalam *Dataset*.....9

BAB III KLASIFIKASI..... 12

3.1. *Split Data*.....12

3.2. Augmentasi Data..... 12

3.3. Skema Klasifikasi.....13

3.4. *Hyperparameter* yang Digunakan..... 14

3.5. Hasil *Training*.....15

3.6. Hasil *Testing*..... 18

3.7. Perbandingan Hasil Evaluasi dan Pembahasan.....25

REFERENSI.....27

DAFTAR GAMBAR

Gambar 1.1 *Malaria Cell Images Dataset* 1

Gambar 3.1 *Confusion Matrix Otsu* 18

Gambar 3.2 *Confusion Matrix Canny* 19

Gambar 3.3 *Confusion Matrix Otsu + Canny* 19

Gambar 3.4 ROC *Curve Otsu* 20

Gambar 3.5 ROC *Curve Canny* 21

Gambar 3.6 ROC *Curve Otsu + Canny* 21

Gambar 3.7 *Learning Curve Otsu* 22

Gambar 3.8 *Learning Curve Canny* 23

Gambar 3.9 *Learning Curve Otsu + Canny* 23

DAFTAR TABEL

Tabel 1.1 Contoh data.....2

Tabel 2.1 *Output Preprocessing*..... 9

Tabel 3.1 Perbandingan waktu *training* dan *testing*..... 24

Tabel 3.2 Perbandingan hasil evaluasi..... 25

BAB I
DATA

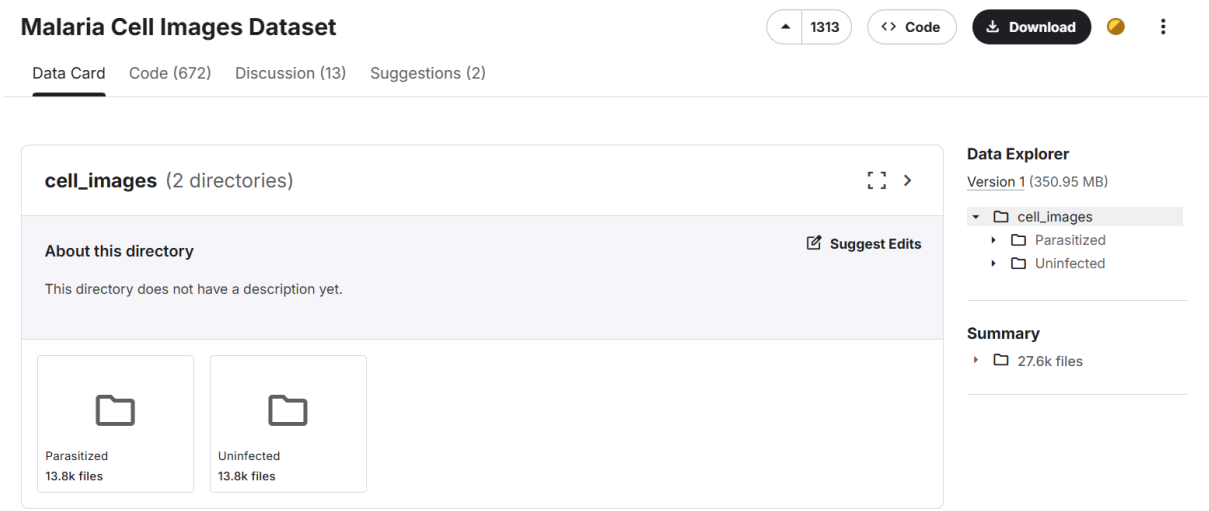
Data yang digunakan dalam penelitian ini merupakan data citra sel darah yang diperoleh dari *platform* Kaggle dengan judul *dataset* "*Malaria Cell Images Dataset*" (https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria) seperti yang ditunjukkan pada Gambar 1.1. *Dataset* ini dikembangkan untuk membantu proses deteksi malaria melalui analisis citra sel darah merah hasil pengamatan mikroskopis. *Dataset* ini terdiri dari sekitar 27.560 citra sel darah merah yang terbagi ke dalam dua kelas, yaitu:

1. *Parasitized* (*infected*)

   Kelas *Parasitized* berisi citra sel darah merah yang terinfeksi parasit malaria.

2. *Uninfected*

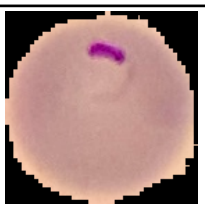
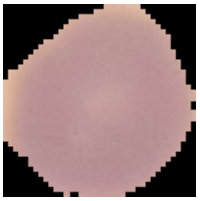
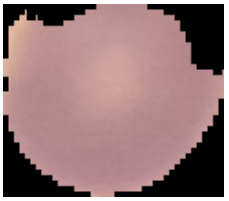
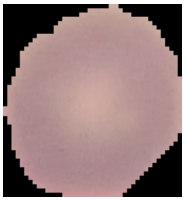
   Kelas *Uninfected* berisi citra sel darah merah dalam kondisi normal tanpa infeksi.



Gambar 1.1 *Malaria Cell Images Dataset*

Jumlah data pada kedua kelas tersebut relatif seimbang, dengan masing-masing kelas terdiri dari 13.780 citra. Setiap data pada *dataset* ini berupa gambar sel darah merah hasil pemotretan mikroskopis yang disimpan dalam format gambar. Contoh data yang digunakan dalam penelitian ini ditunjukkan pada Tabel 1.1. Pada kelas *Parasitized* menunjukkan adanya parasit yang terlihat pada bagian sel darah merah, sedangkan pada kelas *Uninfected* hanya tampak struktur sel darah merah tanpa indikasi infeksi. *Dataset* pada penelitian ini berbentuk citra dengan ukuran yang bervariasi sehingga diperlukan tahap *preprocessing* data sebelum digunakan lebih lanjut.

Tabel 1.1 Contoh data

Kelas	Nama *File*	Gambar
Parasitized	C33P1thinF_IMG_20150619_114756a_cell_179	
	C33P1thinF_IMG_20150619_114756a_cell_180	
	C33P1thinF_IMG_20150619_114756a_cell_181	
Uninfected	C1_thinF_IMG_20150604_104722_cell_9	
	C1_thinF_IMG_20150604_104722_cell_15	
	C1_thinF_IMG_20150604_104722_cell_60	

BAB II
PREPROCESSING

2.1 *Import Libraries*

```
# ============================================================
# IMPORT LIBRARIES
# ============================================================
import cv2
import os
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
```

Code ini digunakan untuk mengimport *library* pendukung dalam proses pengolahan citra digital. *Library* cv2 (*OpenCV*) berfungsi untuk membaca, memproses, dan menyimpan gambar, termasuk penerapan metode seperti *Otsu Thresholding* dan *Canny Edge Detection*. *Library* OS digunakan untuk mengelola operasi sistem *file* seperti membuat atau memeriksa folder. *matplotlib.pyplot* digunakan untuk visualisasi hasil pemrosesan gambar dalam bentuk grafik atau tampilan citra. Sementara itu, *numpy* berguna untuk operasi numerik pada data gambar karena setiap citra direpresentasikan sebagai *array* numerik. Terakhir, *pathlib* digunakan untuk mempermudah pengaturan dan navigasi *path* folder secara modern dan lebih terstruktur dibandingkan dengan penggunaan *os.path*.

2.2 Pengaturan *Path Dataset*

```
# ========================================================
# PENGATURAN PATH DATASET
# ========================================================
# Langsung arahkan ke folder dataset (gunakan raw string untuk menghindari error path)
parasitized_path = Path(r"C:\Documents\Tugas Evita\semester 7\COMVIS\UTS\Code\Parasitized")
uninfected_path = Path(r"C:\Documents\Tugas Evita\semester 7\COMVIS\UTS\Code\Uninfected")
```

Code ini digunakan untuk mengatur lokasi (*path*) *dataset* gambar yang akan diproses. Pada bagian ini, variabel parasitized_path dan uninfected_path masing-masing menyimpan alamat folder yang berisi gambar sel darah terinfeksi (*Parasitized*) dan tidak terinfeksi (*Uninfected*). Fungsi *Path()* dari *library pathlib* digunakan untuk mempermudah manajemen dan manipulasi *path* secara lebih aman dan fleksibel dibandingkan dengan *string* biasa. Tanda r sebelum tanda kutip (r"...") menunjukkan *raw string*, yang digunakan agar Python membaca *path* secara literal tanpa menganggap tanda *backslash* (\) sebagai karakter khusus. Dengan demikian, baris ini memastikan program dapat menemukan dan mengakses *dataset* gambar dengan benar dari lokasi penyimpanan di komputer.

2.3 Pengaturan *Path Output*

```
# ============================================================
# PENGATURAN PATH OUTPUT
# ============================================================
# Folder utama untuk menyimpan hasil pemrosesan
output_path = Path("processed_images")
```

```
# Subfolder berdasarkan kategori
parasitized_output = output_path / "Parasitized"
uninfected_output = output_path / "Uninfected"
```

Code ini digunakan untuk menentukan lokasi penyimpanan hasil pemrosesan gambar. Variabel *output_path* berfungsi sebagai folder utama bernama *processed_images*, yang menjadi tempat semua hasil pemrosesan akan disimpan. Di dalam folder utama tersebut, dibuat dua subfolder tambahan *parasitized_output* untuk menyimpan hasil gambar dari kategori *Parasitized* (terinfeksi), dan *uninfected_output* untuk kategori *Uninfected* (tidak terinfeksi). Penggunaan operator / pada *Path* secara otomatis menggabungkan nama folder sehingga membentuk struktur direktori yang rapi seperti *processed_images/Parasitized* dan *processed_images/Uninfected*. Struktur ini memudahkan pengelolaan hasil pemrosesan agar terpisah sesuai dengan jenis datanya.

2.4 Subfolder untuk Setiap Metode Pemrosesan

```
# ============================================================
# SUBFOLDER UNTUK SETIAP METODE PEMROSESAN
# ============================================================
# Parasitized
parasitized_original = parasitized_output / "original"
parasitized_otsu = parasitized_output / "otsu"
parasitized_canny = parasitized_output / "canny"
parasitized_otsu_canny = parasitized_output / "otsu_canny"

# Uninfected
uninfected_original = uninfected_output / "original"
uninfected_otsu = uninfected_output / "otsu"
uninfected_canny = uninfected_output / "canny"
uninfected_otsu_canny = uninfected_output / "otsu_canny"
```

Code ini digunakan untuk membuat struktur subfolder yang akan menyimpan hasil pemrosesan gambar berdasarkan metode yang digunakan dan kategori datanya. *Dataset* dibagi menjadi dua kategori utama, yaitu *Parasitized* (terinfeksi) dan *Uninfected* (tidak terinfeksi). Masing-masing kategori memiliki empat subfolder untuk menyimpan hasil dari tahapan pemrosesan yang berbeda, yaitu:

a. *original* untuk menyimpan gambar asli sebelum diproses.
b. *otsu* untuk hasil segmentasi menggunakan metode *Otsu Thresholding*.
c. *canny* untuk hasil deteksi tepi menggunakan *Canny Edge Detection*.
d. *otsu_canny* untuk hasil kombinasi antara *Otsu Thresholding* dan *Canny Edge Detection*.

Penggunaan operator / dalam *pathlib* secara otomatis menggabungkan nama folder utama dengan subfoldernya, sehingga menghasilkan struktur direktori yang rapi dan mudah diakses dalam proses penyimpanan hasil pemrosesan.

2.5 Pembuatan Folder Otomatis

```
# ========================================================
# PEMBUATAN FOLDER OTOMATIS (JIKA BELUM ADA)
# ========================================================
for folder in [
    parasitized_original, parasitized_otsu, parasitized_canny, parasitized_otsu_canny,
    uninfected_original, uninfected_otsu, uninfected_canny, uninfected_otsu_canny
]:
    folder.mkdir(parents=True, exist_ok=True)
```

Code ini digunakan untuk membuat seluruh folder *output* secara otomatis jika folder tersebut belum ada di dalam direktori penyimpanan. Melalui perulangan *for*, setiap *path folder* yang sudah didefinisikan sebelumnya (seperti *parasitized_original*, *uninfected_otsu*, dan lainnya) akan diperiksa satu per satu. Fungsi *mkdir(parents=True, exist_ok=True)* dari *library pathlib* berfungsi untuk membuat folder beserta folder induknya (*parents=True*) jika belum ada, serta mencegah *error* jika folder tersebut sudah ada (*exist_ok=True*). *Code* ini memastikan seluruh struktur direktori untuk menyimpan hasil pemrosesan gambar tersedia dan siap digunakan sebelum proses dimulai, tanpa perlu membuatnya secara manual.

2.6 Fungsi untuk Memproses Gambar

```python
# ============================================================
# FUNGSI UNTUK MEMPROSES GAMBAR
# ============================================================
Tabnine | Edit | Test | Explain | Document
def process_image(image_path):
    # 1. Baca gambar grayscale
    image = cv2.imread(str(image_path), cv2.IMREAD_GRAYSCALE)
    if image is None:
        print(f"Gagal membaca gambar: {image_path}")
        return None, None, None, None

    # 2. Otsu Thresholding
    _, otsu = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # 3. Canny Edge Detection (langsung dari grayscale)
    canny = cv2.Canny(image, 100, 200)

    # 4. Canny setelah Otsu (agar hasil berbeda)
    canny_after_otsu = cv2.Canny(otsu, 100, 200)

    return image, otsu, canny, canny_after_otsu
```

Code ini berfungsi untuk melakukan proses pengolahan citra pada setiap gambar yang diambil dari *dataset*. Fungsi *process_image()* menerima parameter *image_path*, yaitu lokasi file gambar yang akan diproses. Langkah-langkahnya sebagai berikut:

a. Membaca gambar dalam format *grayscale* menggunakan *cv2.imread(..., cv2.IMREAD_GRAYSCALE)*. Konversi ke *grayscale* dilakukan agar pemrosesan citra lebih efisien karena hanya menggunakan satu kanal intensitas cahaya, bukan tiga kanal warna (RGB).

b. Pengecekan validitas gambar jika gambar gagal dibaca (misalnya karena *file* rusak atau tidak ditemukan), fungsi akan menampilkan peringatan dan mengembalikan nilai *None*.

c. Menerapkan *Otsu Thresholding*, yaitu metode segmentasi otomatis yang menentukan ambang batas optimal untuk memisahkan objek (*foreground*) dan latar belakang (*background*) berdasarkan distribusi intensitas piksel.

d. Menerapkan *Canny Edge Detection*, yang digunakan untuk mendeteksi tepi-tepi penting dalam gambar dengan dua nilai ambang (100 dan 200) untuk mengontrol sensitivitas deteksi.

e. Menggabungkan hasil *Otsu* dan *Canny*, di mana hasil tepi dari *Canny* kembali diproses menggunakan *threshold* Otsu untuk mendapatkan segmentasi tepi yang lebih tajam dan bersih.

Fungsi ini kemudian menghasilkan empat hasil, yaitu gambar asli (*grayscale*), hasil *Otsu*, hasil *Canny*, dan hasil kombinasi *Otsu* + *Canny*, yang nantinya akan disimpan ke folder masing-masing.

2.7 Fungsi untuk Menyimpan Hasil Pemrosesan

```
# ==========================================================
# FUNGSI UNTUK MENYIMPAN HASIL PEMROSESAN
# ==========================================================
Tabnine | Edit | Test | Explain | Document
def save_results(image_name, image, otsu, canny, otsu_canny, category):
    # Menentukan folder tujuan berdasarkan kategori gambar
    if category == "Parasitized":
        original_folder = parasitized_original
        otsu_folder = parasitized_otsu
        canny_folder = parasitized_canny
        otsu_canny_folder = parasitized_otsu_canny
    else:
        original_folder = uninfected_original
        otsu_folder = uninfected_otsu
        canny_folder = uninfected_canny
        otsu_canny_folder = uninfected_otsu_canny

    # Menyimpan hasil ke folder yang sesuai
    cv2.imwrite(str(original_folder / f"{image_name}.png"), image)
    cv2.imwrite(str(otsu_folder / f"{image_name}_otsu.png"), otsu)
    cv2.imwrite(str(canny_folder / f"{image_name}_canny.png"), canny)
    cv2.imwrite(str(otsu_canny_folder / f"{image_name}_otsu_canny.png"), otsu_canny)
```

Code ini berfungsi untuk menyimpan hasil pemrosesan gambar ke dalam folder yang sesuai berdasarkan kategori dan metode pengolahannya. Fungsi *save_results()* menerima beberapa parameter, yaitu *image_name* sebagai nama *file* gambar, hasil pemrosesan (*image, otsu, canny,* dan *otsu_canny*), serta kategori yang menunjukkan jenis gambar (apakah "*Parasitized*" atau "*Uninfected*"). Pertama, program menentukan lokasi folder penyimpanan berdasarkan kategori gambar. Jika gambar termasuk kategori "*Parasitized*", maka hasilnya disimpan di dalam folder *parasitized_output*; sebaliknya, jika termasuk kategori "*Uninfected*", maka disimpan di dalam folder *uninfected_output*. Setelah itu, setiap hasil pemrosesan disimpan menggunakan fungsi *cv2.imwrite()*, yang berfungsi menyimpan gambar dalam format *.png* ke folder masing-masing yaitu folder original untuk gambar awal, *otsu* untuk hasil segmentasi *otsu*, *canny* untuk hasil deteksi tepi, dan *otsu_canny* untuk hasil kombinasi kedua metode tersebut. Dengan cara ini, seluruh hasil pemrosesan tersimpan secara terorganisir sesuai dengan kategori dan jenis pemrosesannya.

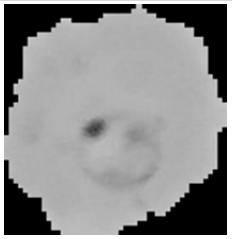
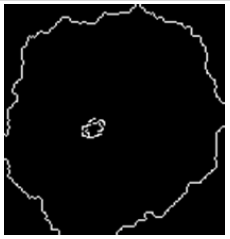
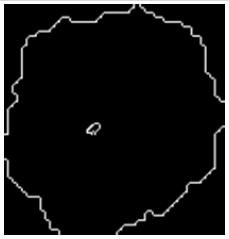
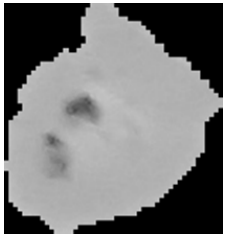
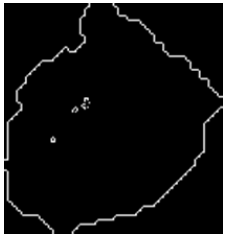
2.8 Pemrosesan Semua Gambar dalam *Dataset*

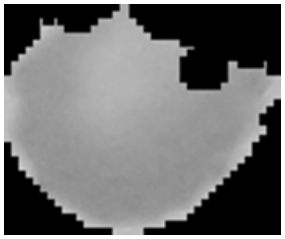
```
# ==========================================================
# PEMROSESAN SEMUA GAMBAR DALAM DATASET
# ==========================================================
# Ambil semua gambar dari dua folder
sample_images = [
    (image_path, "Parasitized") for image_path in parasitized_path.glob("*.png")
] + [
    (image_path, "Uninfected") for image_path in uninfected_path.glob("*.png")
]

# Melakukan pemrosesan untuk setiap gambar
for image_path, category in sample_images:
    print(f"Processing: {image_path.name} ({category})")
    image, otsu, canny, otsu_canny = process_image(image_path)

    if image is not None:  # Pastikan gambar terbaca
        save_results(image_path.stem, image, otsu, canny, otsu_canny, category)

print("\nSemua gambar berhasil diproses dan disimpan di folder 'processed_images'.")
```

Code ini berfungsi untuk memproses seluruh gambar dari dua folder *dataset*, yaitu *Parasitized* dan *Uninfected*. Baris pertama mengambil semua *file* berformat *.png* dari kedua folder tersebut dan menyimpannya dalam daftar *sample_images* beserta kategorinya. Setiap gambar kemudian diproses satu per satu menggunakan fungsi *process_image()* yang menghasilkan empat *output*, yaitu gambar asli, hasil *Otsu*, *Canny*, dan gabungan keduanya. Jika gambar berhasil dibaca, hasil pemrosesan disimpan melalui fungsi *save_results()* ke folder sesuai kategorinya. Bagian *print()* digunakan untuk menampilkan nama gambar yang sedang diproses agar memudahkan pemantauan. Setelah semua gambar selesai diproses, program menampilkan pesan bahwa seluruh hasil telah disimpan di folder *processed_images*. Struktur ini memastikan proses berjalan otomatis dan hasil tersimpan rapi berdasarkan jenis datanya.

2.9 *Output* Pemrosesan Semua Gambar dalam *Dataset*

Tabel 2.1 *Output Preprocessing*

Folder (Kelas)	Sampel Gambar	*Grayscale*	*Otsu*	*Canny*	*Otsu* + *Canny*
Parasitized	**Gambar 1:**				
	Gambar 2:				
	Gambar 3:				

Folder (Kelas)	Sampel Gambar	*Grayscale*	*Otsu*	*Canny*	*Otsu + Canny*
Uninfected	**Gambar 1:**				
	Gambar 2:				
	Gambar 3:				

Tabel 2.1 menampilkan hasil dari seluruh proses pengolahan citra pada *dataset* sel darah yang terbagi menjadi dua kelas atau kategori, yaitu *Parasitized* (terinfeksi) dan *Uninfected* (tidak terinfeksi). Setiap gambar diproses menggunakan tiga metode utama, yaitu *Otsu Thresholding*, *Canny Edge Detection*, dan kombinasi *Otsu* + *Canny*. Gambar *Grayscale* menunjukkan citra awal dalam bentuk *grayscale* sebelum dilakukan pemrosesan. Metode *Otsu* menghasilkan segmentasi otomatis yang memisahkan objek utama dari latar belakang berdasarkan intensitas piksel, sedangkan metode *Canny* berfungsi untuk menampilkan tepi-tepi penting pada gambar. Hasil kombinasi *Otsu* + *Canny* memberikan tampilan yang lebih tajam dan bersih karena menggabungkan kemampuan segmentasi *Otsu* dengan deteksi tepi *Canny*.

Pada gambar *Parasitized*, hasil *Otsu* membantu menyorot area yang terinfeksi parasit di dalam sel darah, sementara metode *Canny* menampilkan kontur atau batas sel dengan lebih jelas. Kombinasi keduanya menghasilkan citra dengan detail yang lebih kuat dan memudahkan dalam membedakan bagian yang terinfeksi. Sementara itu, pada gambar *Uninfected*, hasil *Otsu* tampak lebih merata tanpa area gelap yang dominan, dan hasil *Canny* memperlihatkan struktur sel yang normal dengan batas tepi yang halus. Dengan demikian, penerapan metode *Otsu*, *Canny*, dan kombinasi keduanya mampu menghasilkan citra yang lebih informatif dan membantu dalam proses identifikasi antara sel darah yang terinfeksi dan tidak.

BAB III
KLASIFIKASI

3.1. *Split* Data

```
class_names = full_dataset.classes
print("Classes:", class_names)

total_len = len(full_dataset)
train_len = int(0.7 * total_len)
val_len = int(0.15 * total_len)
test_len = total_len - train_len - val_len

train_indices, val_indices, test_indices = torch.utils.data.random_split(
    range(total_len), [train_len, val_len, test_len]
)
```

Pada penelitian ini, *dataset* dibagi menjadi tiga subset, yaitu data *training* (70%), data *validation* (15%), dan data *testing* (15%). Pembagian data dilakukan secara acak menggunakan fungsi '*random_split*' pada *PyTorch*. Data *training* digunakan untuk melatih model, data *validation* digunakan untuk memantau kinerja model selama proses pelatihan serta menentukan model terbaik, sedangkan data *testing* digunakan untuk mengevaluasi performa akhir model setelah proses *training* selesai.

3.2. Augmentasi Data

```
# ============================================================
# DATA AUGMENTATION
# ============================================================
train_transform = transforms.Compose([
    transforms.Grayscale(3),
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.15, contrast=0.15),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.15))
])

val_transform = transforms.Compose([
    transforms.Grayscale(3),
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])

def filter_preprocess(path):
    return PREPROCESS_MODE in path
```

Teknik augmentasi data diterapkan hanya pada data *training* untuk meningkatkan variasi data dan mengurangi risiko *overfitting*. Augmentasi yang digunakan meliputi rotasi acak, pembalikan *horizontal*, penyesuaian kecerahan dan kontras, serta penghapusan acak sebagian area citra (*random erasing*). Sementara itu, data *validation* dan data *testing* tidak diberikan augmentasi agar tetap merepresentasikan kondisi data asli. Setiap citra terlebih dahulu dikonversi menjadi tiga kanal menggunakan '*Grayscale(3)*' dan diubah ukurannya menjadi 224×224 piksel agar sesuai dengan input model *EfficientNetV2-S*.

Beberapa transformasi acak diterapkan untuk memperkaya variasi citra, antara lain pembalikan horizontal secara acak (*RandomHorizontalFlip*), rotasi dengan sudut kecil (*RandomRotation*), serta penyesuaian kecerahan dan kontras menggunakan '*ColorJitter*'. Transformasi ini bertujuan untuk membuat model lebih *robust* terhadap variasi orientasi dan kondisi pencahayaan citra. Setelah proses augmentasi geometris dan warna, citra diubah ke dalam bentuk *tensor* menggunakan '*ToTensor*()'. Selain itu, diterapkan teknik '*RandomErasing*' untuk menghapus sebagian area citra secara acak, yang bertujuan mengurangi ketergantungan model pada fitur tertentu dan menurunkan risiko *overfitting*. Data *validation* dan data *testing* tidak diberikan augmentasi agar tetap merepresentasikan kondisi data asli.

3.3. Skema Klasifikasi

```
# ============================================================
# MODEL DENGAN DROPOUT
# ============================================================
print("\nLoading EfficientNetV2-S...")
weights = EfficientNet_V2_S_Weights.IMAGENET1K_V1
model = efficientnet_v2_s(weights=weights)

# Modifikasi classifier dengan dropout
num_features = model.classifier[1].in_features
model.classifier = nn.Sequential(
    nn.Dropout(p=0.3, inplace=True),
    nn.Linear(num_features, 512),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.4),
    nn.Linear(512, len(class_names))
)

model = model.to(device)
print("Model loaded successfully!")
```

Skema klasifikasi pada penelitian ini menggunakan arsitektur *EfficientNetV2-S* dengan bobot *pre-training ImageNet*. Model diinisialisasi menggunakan *pretrained weights* untuk memanfaatkan fitur visual dasar yang telah dipelajari sebelumnya. Bagian *classifier*

pada EfficientNetV2-S dimodifikasi agar sesuai dengan *dataset* penelitian. Lapisan *dropout* digunakan untuk mengurangi risiko *overfitting* dengan menonaktifkan sebagian neuron secara acak selama proses pelatihan.

Lapisan *fully connected* (Linear) berfungsi untuk memetakan fitur hasil ekstraksi ke ruang kelas yang lebih kecil, sedangkan fungsi aktivasi ReLU digunakan untuk memperkenalkan non-linearitas. Lapisan *output* disesuaikan dengan jumlah kelas menggunakan '*len(class_names)*' sehingga model dapat melakukan klasifikasi sesuai dengan kategori yang digunakan. Model kemudian dipindahkan ke perangkat komputasi (CPU/GPU) untuk memastikan proses *training* dan *testing* dapat berjalan secara efisien.

3.4. *Hyperparameter* yang Digunakan

```
# ============================================================
# KONFIGURASI
# ============================================================
DATA_ROOT = "/kaggle/input/processed-images/processed_images"
PREPROCESS_MODE = "otsu_canny"
BATCH_SIZE = 64
LEARNING_RATE = 5e-5
WEIGHT_DECAY = 1e-4
PATIENCE = 5
MAX_EPOCHS = 30

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

Pada tahap konfigurasi awal, ditentukan beberapa parameter utama yang digunakan dalam proses pelatihan model. Nilai '*batch_size*' ditetapkan sebesar 64, yang menunjukkan jumlah data yang diproses dalam satu iterasi pelatihan. Parameter '*learning_rate*' sebesar 0,00005 digunakan untuk mengatur besar pembaruan bobot model, sedangkan '*weight_decay*' sebesar 0,0001 berfungsi sebagai regularisasi untuk mengurangi risiko *overfitting*. Proses pelatihan dijalankan dengan maksimum 30 *epoch* dan dilengkapi mekanisme *early stopping* dengan nilai *patience* sebesar 5 untuk menghentikan pelatihan apabila kinerja model tidak mengalami peningkatan. Selanjutnya, pemilihan perangkat komputasi dilakukan secara otomatis dengan memanfaatkan GPU apabila tersedia, atau CPU jika GPU tidak tersedia. Langkah ini bertujuan untuk meningkatkan efisiensi komputasi selama proses *training* dan *testing* model.

```python
# ============================================================
# LOSS & OPTIMIZER
# ============================================================
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=3
)
```

Pada tahap optimasi, fungsi *loss* yang digunakan adalah *cross entropy loss* dengan label *smoothing* sebesar 0,1. Penggunaan label *smoothing* bertujuan untuk meningkatkan stabilitas pelatihan dengan mengurangi tingkat kepercayaan model terhadap prediksi tertentu. Proses pembaruan bobot dilakukan menggunakan algoritma *AdamW* dengan *learning rate* dan *weight decay* yang telah ditentukan sebelumnya. Untuk menjaga kestabilan proses pelatihan, digunakan *learning rate scheduler* '*ReduceLROnPlateau*'. *Scheduler* ini menurunkan nilai *learning rate* secara otomatis ketika performa model tidak menunjukkan peningkatan dalam beberapa *epoch* sehingga membantu proses konvergensi model menjadi lebih optimal.

3.5. Hasil *Training*

Hasil *training* untuk semua skema klasifikasi.

1. Hasil *Training Otsu*

```
Epoch 17/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.3053, acc=0.9363]
Val: 100%|██████████| 65/65 [00:13<00:00,  4.77it/s, loss=0.3815, acc=0.8961]
Train Loss: 0.3053 | Train Acc: 0.9363
Val   Loss: 0.3815 | Val   Acc: 0.8961
Gap: 0.0402 | LR: 2.50e-05 | Time: 251.1s
✓ Saved! Best Val Acc: 0.8961

Epoch 18/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.2984, acc=0.9421]
Val: 100%|██████████| 65/65 [00:13<00:00,  4.76it/s, loss=0.3934, acc=0.8909]
Train Loss: 0.2984 | Train Acc: 0.9421
Val   Loss: 0.3934 | Val   Acc: 0.8909
Gap: 0.0513 | LR: 2.50e-05 | Time: 251.1s
✗ No improvement (1/5)
```

```
Epoch 21/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.2862, acc=0.9488]
Val: 100%|██████████| 65/65 [00:13<00:00,  4.94it/s, loss=0.3980, acc=0.8957]
Train Loss: 0.2862 | Train Acc: 0.9488
Val   Loss: 0.3980 | Val   Acc: 0.8957
Gap: 0.0531 | LR: 1.25e-05 | Time: 250.5s
✗ No improvement (4/5)

Epoch 22/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.2789, acc=0.9541]
Val: 100%|██████████| 65/65 [00:13<00:00,  4.96it/s, loss=0.3942, acc=0.8951]
Train Loss: 0.2789 | Train Acc: 0.9541
Val   Loss: 0.3942 | Val   Acc: 0.8951
Gap: 0.0590 | LR: 1.25e-05 | Time: 250.5s
✗ No improvement (5/5)

 EARLY STOPPING

============================================================
Training Done: 92.6 min (5554.4s)
Best Val Acc: 0.8961
============================================================
```

Hasil *training* dengan *pre-processing Otsu* menunjukkan bahwa model mencapai performa terbaik pada *epoch* ke-17 dengan nilai akurasi validasi sebesar 0,8961. Setelah *epoch* tersebut, tidak terjadi peningkatan akurasi validasi meskipun proses *training* dilanjutkan. Oleh karena itu, mekanisme *early stopping* diaktifkan pada epoch ke-22 karena kinerja validasi tidak mengalami perbaikan selama lima *epoch* berturut-turut sehingga *training* dihentikan untuk mencegah *overfitting* dan menghemat waktu komputasi. Total waktu *training* yang dibutuhkan adalah sekitar 92,6 menit. Pada *epoch* terakhir (*epoch* ke-22), model menghasilkan *train loss* sebesar 0,2789 dan *validation loss* sebesar 0,3942, sedangkan pada *epoch* terbaik (*epoch* ke-17) diperoleh *train loss* sebesar 0,3053 dan *validation loss* sebesar 0,3815, yang menunjukkan bahwa peningkatan performa *training* setelah *epoch* terbaik tidak diikuti oleh peningkatan kinerja validasi.

2. Hasil *Training Canny*

```
Val   Loss: 0.2764 | Val   Acc: 0.9631
Gap: 0.0178 | LR: 5.00e-05 | Time: 253.5s
✓ Saved! Best Val Acc: 0.9631

Epoch 29/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.2321, acc=0.9822]
Val: 100%|██████████| 65/65 [00:13<00:00,  4.64it/s, loss=0.2798, acc=0.9625]
Train Loss: 0.2321 | Train Acc: 0.9822
Val   Loss: 0.2798 | Val   Acc: 0.9625
Gap: 0.0197 | LR: 5.00e-05 | Time: 251.5s
✗ No improvement (1/5)

Epoch 30/30
----------------------------------------
Train: 100%|██████████| 603/603 [03:57<00:00,  2.54it/s, loss=0.2306, acc=0.9835]
Val: 100%|██████████| 65/65 [00:16<00:00,  3.87it/s, loss=0.2728, acc=0.9667]
Train Loss: 0.2306 | Train Acc: 0.9835
Val   Loss: 0.2728 | Val   Acc: 0.9667
Gap: 0.0168 | LR: 5.00e-05 | Time: 254.2s
✓ Saved! Best Val Acc: 0.9667

============================================================
Training Done: 125.6 min (7538.9s)
Best Val Acc: 0.9667
============================================================
```

Hasil *training* dengan *pre-processing Canny* menunjukkan bahwa akurasi validasi terbaik dicapai pada *epoch* ke-30 dengan nilai 0,9667 sehingga bobot model terbaik disimpan pada *epoch* tersebut. Proses *training* dijalankan hingga mencapai epoch maksimum tanpa penerapan *early stopping* karena kinerja validasi masih mengalami peningkatan hingga *epoch* terakhir. Total waktu *training* yang dibutuhkan adalah sekitar 125,6 menit. Pada *epoch* terbaik (*epoch* ke-30), model menghasilkan *train loss* sebesar 0,2306 dan *validation loss* sebesar 0,2728, dengan *train accuracy* sebesar 0,9835 dan *validation accuracy* sebesar 0,9667, yang menunjukkan bahwa model mampu belajar secara optimal dan memiliki kemampuan generalisasi yang baik.

3. Hasil *Training Otsu + Canny*

```
Epoch 8/30
----------------------------------------
Train: 100%|██████████| 302/302 [01:58<00:00,  2.54it/s, loss=0.3433, acc=0.9147]
Val: 100%|██████████| 33/33 [00:06<00:00,  4.79it/s, loss=0.3447, acc=0.9148]
Train Loss: 0.3433 | Train Acc: 0.9147
Val   Loss: 0.3447 | Val   Acc: 0.9148
Gap: -0.0002 | LR: 5.00e-05 | Time: 125.8s
✓ Saved! Best Val Acc: 0.9148

Epoch 9/30
----------------------------------------
Train: 100%|██████████| 302/302 [01:58<00:00,  2.54it/s, loss=0.3377, acc=0.9181]
Val: 100%|██████████| 33/33 [00:10<00:00,  3.21it/s, loss=0.3494, acc=0.9100]
Train Loss: 0.3377 | Train Acc: 0.9181
Val   Loss: 0.3494 | Val   Acc: 0.9100
Gap: 0.0081 | LR: 5.00e-05 | Time: 129.2s
✗ No improvement (1/5)
```

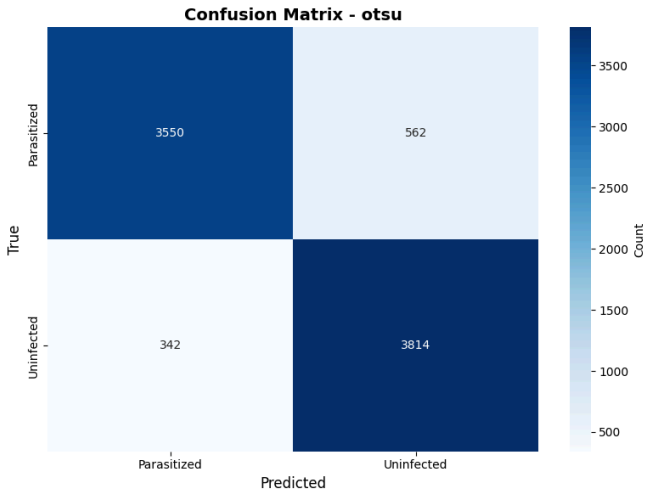
```
Epoch 12/30
----------------------------------------
Train: 100%|██████████| 302/302 [01:58<00:00,  2.54it/s, loss=0.3253, acc=0.9256]
Val: 100%|██████████| 33/33 [00:07<00:00,  4.55it/s, loss=0.3517, acc=0.9098]
Train Loss: 0.3253 | Train Acc: 0.9256
Val   Loss: 0.3517 | Val   Acc: 0.9098
Gap: 0.0158 | LR: 2.50e-05 | Time: 126.2s
✗ No improvement (4/5)

Epoch 13/30
----------------------------------------
Train: 100%|██████████| 302/302 [02:01<00:00,  2.49it/s, loss=0.3114, acc=0.9349]
Val: 100%|██████████| 33/33 [00:06<00:00,  4.83it/s, loss=0.3599, acc=0.9066]
Train Loss: 0.3114 | Train Acc: 0.9349
Val   Loss: 0.3599 | Val   Acc: 0.9066
Gap: 0.0283 | LR: 2.50e-05 | Time: 127.9s
✗ No improvement (5/5)

 EARLY STOPPING

============================================================
Training Done: 27.4 min (1645.0s)
Best Val Acc: 0.9148
============================================================
```

Hasil *training* dengan *pre-processing Otsu + Canny* menunjukkan bahwa akurasi validasi terbaik dicapai pada *epoch* ke-8 dengan nilai 0,9148 sehingga bobot model terbaik disimpan pada *epoch* tersebut. Proses *training* kemudian dilanjutkan hingga *epoch* ke-13, namun dihentikan melalui mekanisme *early stopping* karena tidak terjadi peningkatan akurasi validasi selama lima *epoch* berturut-turut. Total waktu *training* yang dibutuhkan adalah sekitar 27,4 menit. Pada *epoch* terakhir (*epoch* ke-13), model menghasilkan *train loss* sebesar 0,3114 dan *validation loss* sebesar 0,3599, sedangkan *epoch* terbaik (*epoch* ke-8), model menghasilkan *train loss* sebesar 0,3433 dan *validation loss* sebesar 0,3447, dengan *train accuracy* sebesar 0,9147 dan *validation accuracy* sebesar 0,9148, yang menunjukkan keseimbangan kinerja antara data latih dan data validasi.

3.6. Hasil *Testing*

Hasil *testing* untuk semua skema klasifikasi.

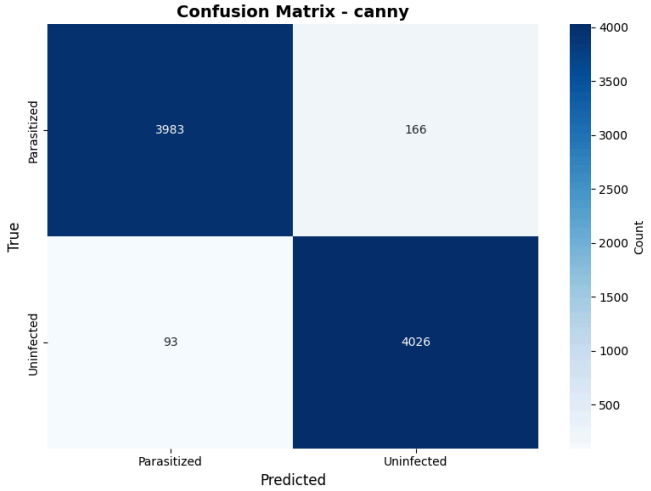
1. *Confusion Matrix Otsu*



Gambar 3.1 *Confusion Matrix Otsu*

Berdasarkan Gambar 3.1, model mampu mengklasifikasikan data dengan cukup baik, ditunjukkan oleh jumlah prediksi benar yang relatif tinggi pada kedua kelas. Terdapat 3.550 data Parasitized yang berhasil diklasifikasikan dengan benar dan 3.814 data *Uninfected* yang juga teridentifikasi dengan tepat. Namun demikian, masih terdapat kesalahan klasifikasi, yaitu 562 data *Parasitized* yang keliru diprediksi sebagai *Uninfected* serta 342 data *Uninfected* yang salah diklasifikasikan sebagai *Parasitized*. Hal ini menunjukkan bahwa meskipun metode *Otsu* cukup efektif dalam memisahkan kedua kelas, tingkat kesalahan masih cukup signifikan, terutama pada kasus parasit yang tidak terdeteksi.

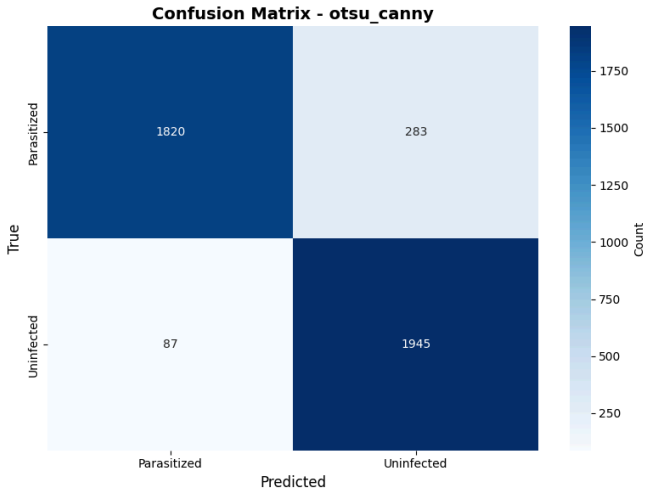
2. *Confusion Matrix Canny*



Gambar 3.2 *Confusion Matrix Canny*

Gambar 3.2 menunjukkan performa klasifikasi yang lebih baik dibandingkan *Otsu*. Model berhasil mengklasifikasikan 3.983 data *Parasitized* dan 4.026 data *Uninfected* secara benar, dengan jumlah kesalahan yang relatif kecil. Kesalahan klasifikasi hanya terjadi pada 166 data *Parasitized* yang diprediksi sebagai *Uninfected* dan 93 data *Uninfected* yang salah dikenali sebagai *Parasitized*. Hasil ini mengindikasikan bahwa ekstraksi tepi menggunakan metode *Canny* mampu mempertahankan informasi penting pada citra sehingga meningkatkan kemampuan model dalam membedakan sel terinfeksi dan tidak terinfeksi.

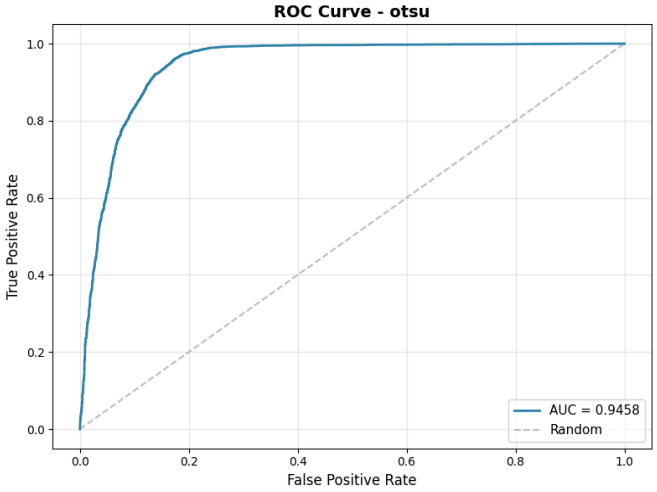
3. *Confusion Matrix Otsu + Canny*



Gambar 3.3 *Confusion Matrix Otsu + Canny*

Pada Gambar 3.3, *confusion matrix* menunjukkan performa yang cukup baik, namun belum melampaui metode *Canny* secara tunggal. Model berhasil mengklasifikasikan 1.820 data *Parasitized* dan 1.945 data *Uninfected* dengan benar. Meskipun demikian, masih terdapat 283 data *Parasitized* yang salah diprediksi sebagai *Uninfected* serta 87 data *Uninfected* yang salah diklasifikasikan sebagai *Parasitized*.

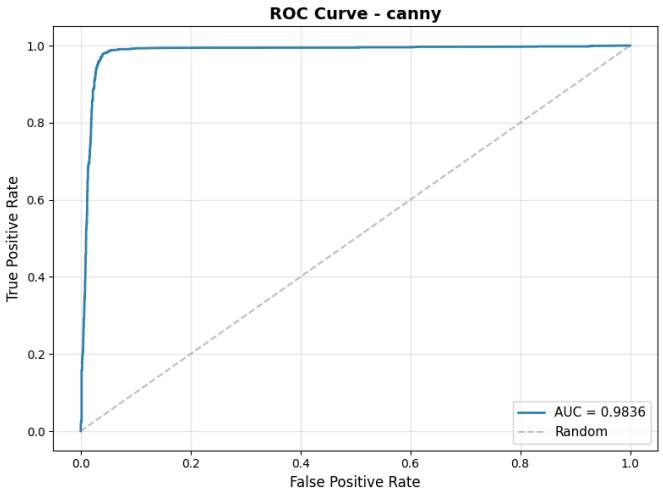
4. ROC *Curve Otsu*



Gambar 3.4 ROC *Curve Otsu*

Dari Gambar 3.4, dapat diketahui kurva ROC untuk metode *Otsu* menghasilkan nilai AUC sebesar 0,9458, yang masih tergolong sangat baik meskipun lebih rendah dibandingkan metode *Canny* dan *Otsu* + *Canny*. Bentuk kurva menunjukkan bahwa metode *Otsu* mampu meningkatkan *True Positive Rate* secara konsisten seiring bertambahnya *False Positive Rate.*

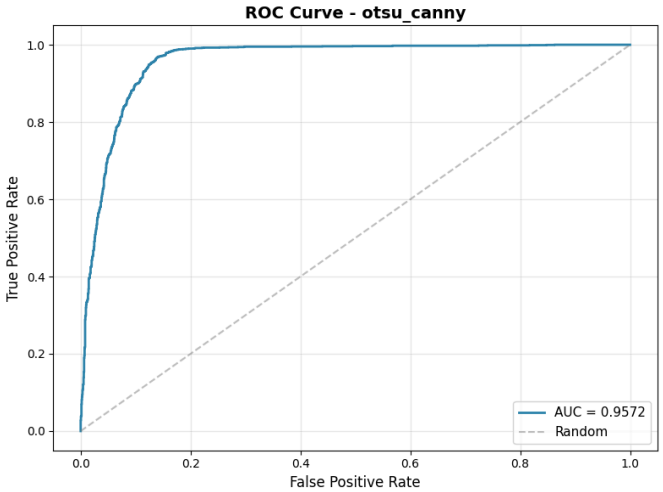
5. ROC *Curve Canny*



Gambar 3.5 ROC *Curve Canny*

Dari Gambar 3.5, dapat diketahui kurva ROC pada metode *Canny* memperlihatkan performa yang paling unggul dibandingkan metode lainnya, dengan nilai AUC sebesar 0,9836. Nilai ini mengindikasikan kemampuan diskriminasi yang sangat tinggi antara kelas positif dan negatif. Kurva yang hampir menempel pada sudut kiri atas grafik menunjukkan bahwa metode *Canny* menghasilkan *True Positive Rate* yang sangat tinggi bahkan pada tingkat *False Positive Rate* yang sangat rendah. Dengan demikian, metode *Canny* terbukti sangat efektif dan stabil dalam mendeteksi fitur yang relevan pada citra sehingga menghasilkan akurasi klasifikasi yang optimal.

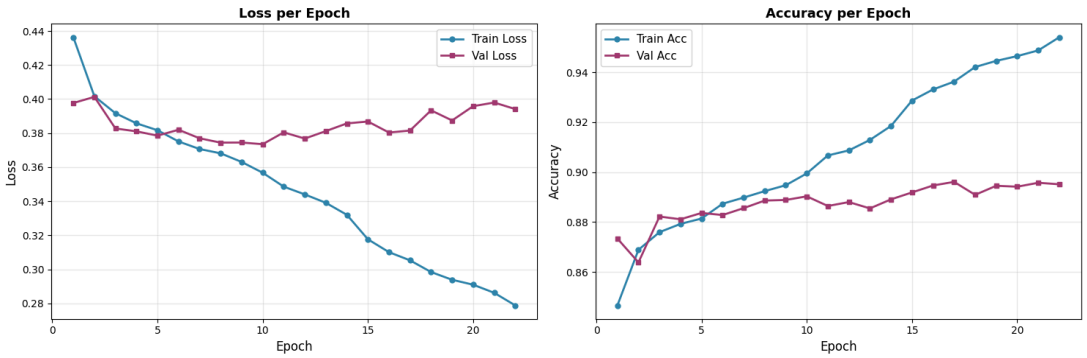
6. ROC *Curve Otsu + Canny*



Gambar 3.6 ROC *Curve Otsu + Canny*

Dari Gambar 3.6, dapat diketahui kurva ROC untuk metode *Otsu* + *Canny* menunjukkan kinerja klasifikasi yang sangat baik dengan nilai AUC sebesar 0,9572. Nilai AUC yang mendekati 1 menandakan bahwa metode ini mampu membedakan kelas positif dan negatif secara efektif. Bentuk kurva yang cepat naik ke arah *True Positive Rate* (TPR) tinggi pada *False Positive Rate* (FPR) yang rendah mengindikasikan bahwa kombinasi *thresholding Otsu* dan deteksi tepi *Canny* dapat mengurangi kesalahan klasifikasi positif palsu sekaligus mempertahankan sensitivitas yang tinggi. Hal ini menunjukkan bahwa metode *Otsu* + *Canny* cukup andal dalam konteks pemrosesan citra.

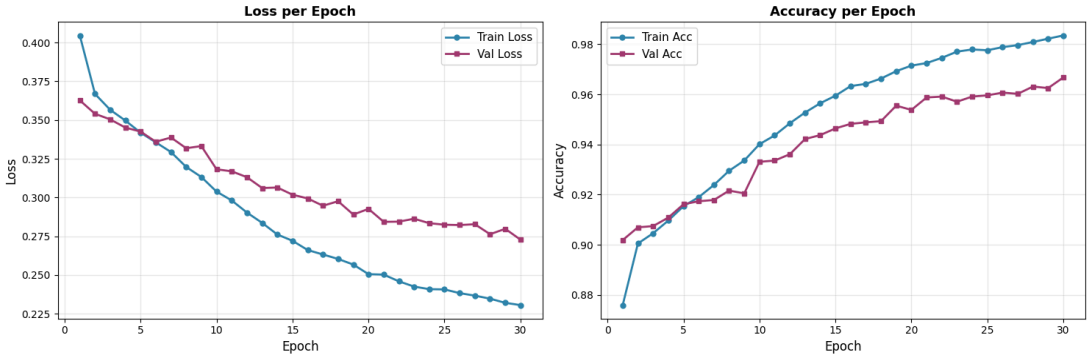
7. *Accuracy Loss Otsu*



Gambar 3.7 *Learning Curve Otsu*

Gambar 3.7 menunjukkan bahwa selama proses pelatihan terjadi peningkatan kinerja model secara konsisten pada data latih, yang ditandai dengan penurunan training *loss* dan kenaikan training *accuracy* di setiap *epoch*. Namun, pada data validasi terlihat bahwa *validation loss* cenderung stagnan bahkan sedikit meningkat setelah beberapa *epoch*, sementara *validation accuracy* hanya mengalami peningkatan kecil dan relatif stabil di kisaran 0,88–0,89. Pola ini mengindikasikan terjadinya *overfitting*, di mana model terlalu menyesuaikan diri dengan karakteristik data latih. Salah satu penyebab utama *overfitting* pada metode *Otsu* adalah sifatnya yang hanya mengandalkan segmentasi berbasis ambang intensitas global sehingga fitur yang dihasilkan cenderung kurang kaya dan kurang representatif terhadap variasi kompleks pada citra sel darah. Akibatnya, model dapat menghafal pola spesifik pada data latih, tetapi kesulitan melakukan generalisasi pada data validasi. Selain itu, penggunaan arsitektur *deep learning* yang cukup kompleks (*EfficientNetV2-S*) dengan fitur input yang relatif sederhana dari hasil *Otsu* juga memperbesar risiko *overfitting*.

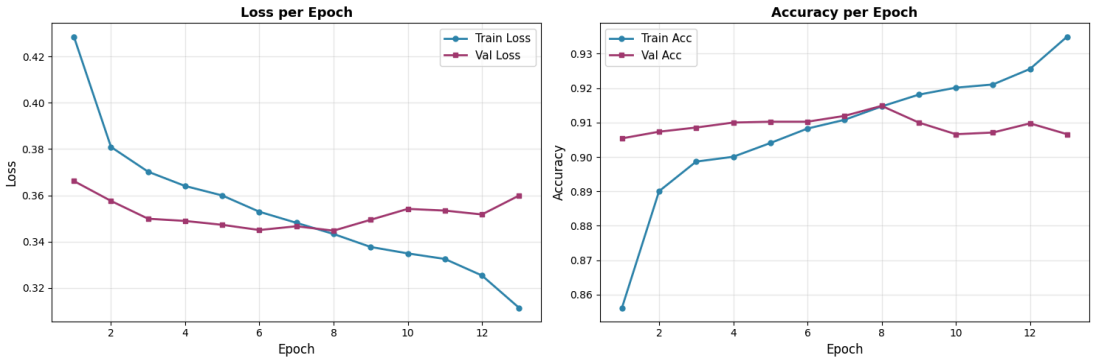
8. *Accuracy Loss Canny*



Gambar 3.8 *Learning Curve Canny*

Gambar 3.8 menunjukkan proses pelatihan model yang berjalan stabil dan efektif baik pada data latih maupun data validasi. *Training loss* dan *validation loss* sama-sama mengalami penurunan yang konsisten seiring bertambahnya *epoch*, tanpa adanya kenaikan signifikan pada *validation loss*, yang menandakan bahwa model tidak mengalami *overfitting* yang berarti. Selisih antara *training loss* dan *validation loss* relatif kecil, menunjukkan kemampuan generalisasi model yang baik. Hal ini juga tercermin pada grafik *accuracy*, di mana *training accuracy* dan *validation accuracy* sama-sama meningkat secara bertahap hingga mencapai nilai tinggi, dengan *validation accuracy* mendekati *training accuracy* di akhir pelatihan. Kondisi ini mengindikasikan bahwa fitur tepi yang dihasilkan oleh metode *Canny* mampu merepresentasikan karakteristik penting citra sel darah secara lebih informatif dan konsisten sehingga membantu model mempelajari pola yang relevan tanpa terlalu menghafal data latih.

9. *Accuracy Loss Otsu + Canny*



Gambar 3.9 *Learning Curve Otsu + Canny*

Gambar 3.9 menunjukkan bahwa model mampu mempelajari pola pada data latih dengan cukup baik, yang ditandai oleh penurunan *training loss* dan peningkatan *training accuracy* secara konsisten di setiap *epoch*. Namun, pada data validasi terlihat bahwa *validation loss* cenderung menurun di awal pelatihan kemudian stagnan dan sedikit meningkat pada *epoch* selanjutnya, sementara *validation accuracy* hanya meningkat hingga titik tertentu lalu dan cenderung stabil di kisaran 0,90–0,91. Pola ini mengindikasikan bahwa meskipun kombinasi *Otsu* dan *Canny* menghasilkan fitur yang lebih kaya dibandingkan *Otsu* saja, model mulai menunjukkan gejala *overfitting* ringan setelah beberapa *epoch*, di mana peningkatan performa pada data latih tidak sepenuhnya diikuti oleh data validasi. Hal ini dapat disebabkan oleh adanya informasi redundan atau *noise* dari penggabungan dua metode *preprocessing*, sehingga sebagian fitur yang dipelajari model kurang berkontribusi terhadap peningkatan kemampuan generalisasi. Secara keseluruhan, metode *Otsu* + *Canny* memberikan performa yang lebih stabil dibandingkan *Otsu* tunggal, tetapi masih berada di bawah metode *Canny* dalam hal konsistensi generalisasi pada data validasi.

10. Perbandingan waktu *training* dan *testing*

Tabel 3.1 Perbandingan waktu *training dan testing*

Skema	Waktu *Training*	Waktu *Testing*
Otsu	92.6 min	16.3 sec
Canny	125.6 min	16.5 sec
Otsu + *Canny*	27.4 min	8.2 sec

Berdasarkan tabel 3.1, skema *Otsu* memiliki waktu *training* sebesar 92,6 menit dan waktu testing 16,3 detik. Waktu *training* yang relatif sedang menunjukkan bahwa proses pembelajaran model dengan hasil segmentasi *Otsu* masih memerlukan komputasi yang cukup besar, namun tidak sekompleks metode *Canny*. Sementara itu, waktu *testing* yang relatif singkat menandakan bahwa model mampu melakukan klasifikasi dengan cukup efisien pada data baru, sehingga masih layak digunakan pada skenario pengujian dengan kebutuhan waktu respons yang tidak terlalu ketat.

Skema *Canny* menunjukkan waktu *training* paling lama, yaitu 125,6 menit, dengan waktu testing sebesar 16,5 detik. Lamanya waktu training mengindikasikan bahwa ekstraksi fitur berbasis tepi pada metode *Canny* menghasilkan representasi data yang lebih kompleks sehingga membutuhkan proses pembelajaran yang lebih intensif.

Meskipun demikian, waktu *testing* hampir setara dengan *Otsu*, yang menunjukkan bahwa kompleksitas tersebut lebih banyak memengaruhi tahap *training* dibandingkan tahap *testing*.

Pada skema kombinasi *Otsu* dan *Canny*, waktu *training* tercatat paling singkat, yaitu 27,4 menit, serta waktu testing tercepat sebesar 8,2 detik. Hasil ini menunjukkan bahwa penggabungan kedua metode mampu menyederhanakan representasi fitur yang digunakan model sehingga proses *training* dan *testing* menjadi lebih efisien

3.7. Perbandingan Hasil Evaluasi dan Pembahasan

Tabel 3.2 Perbandingan hasil evaluasi

Skema	Akurasi	Presisi	Recall	F1-Score	Waktu Komputasi
Otsu	0.8907	0.8716	0.9177	0.8940	93.4 min
Canny	0.9687	0.9604	0.9774	0.9688	126.9 min
Otsu + Canny	0.9105	0.8730	0.9572	0.9131	28.8 min

Berdasarkan Tabel 3.2, dapat dilihat bahwa metode *Otsu*, *Canny*, serta kombinasi *Otsu* dan *Canny* menunjukkan perbedaan kinerja yang signifikan baik dari sisi *accuracy*, *precision*, *recall*, *F1-Score*, maupun waktu komputasi. Perbedaan ini dipengaruhi oleh karakteristik dasar masing-masing metode dalam melakukan segmentasi dan deteksi tepi pada citra digital.

Metode *Otsu* menghasilkan nilai akurasi sebesar 0.8907 dengan nilai *recall* yang cukup tinggi, yaitu 0.9177. Hal ini sejalan dengan karakteristik metode *Otsu* yang menggunakan pendekatan *thresholding global* berdasarkan distribusi histogram citra, sehingga efektif untuk memisahkan objek dan latar belakang pada citra dengan kontras yang jelas [1]. Namun, metode ini memiliki keterbatasan dalam menangani citra dengan variasi intensitas yang kompleks, yang dapat berdampak pada penurunan presisi dan akurasi secara keseluruhan [2].

Metode *Canny* menunjukkan performa paling optimal dengan nilai *accuracy*, *precision*, *recall*, dan *F1-Score* tertinggi dibandingkan metode lainnya. Hasil ini sejalan dengan karakteristik metode *Canny* yang dirancang untuk menghasilkan deteksi tepi yang presisi melalui beberapa tahapan pemrosesan citra [3]. Algoritma *Canny* menerapkan beberapa tahapan penting seperti peredaman *noise* menggunakan *Gaussian filter*, perhitungan gradien, serta *non-maximum suppression*, yang membuat deteksi tepi menjadi lebih presisi dan stabil [4]. Meskipun demikian, kompleksitas tahapan tersebut menyebabkan waktu

komputasi metode *Canny* menjadi lebih lama dibandingkan metode *Otsu* maupun skema kombinasi [5].

Sementara itu, skema kombinasi *Otsu* dan *Canny* menghasilkan waktu komputasi paling cepat, yaitu 28,8 menit, dibandingkan metode *Otsu* sebesar 93,4 menit dan metode *Canny* sebesar 126,9 menit. Dari sisi kinerja, kombinasi ini memperoleh nilai akurasi 0,9105 dengan nilai *recall* yang cukup tinggi, yaitu 0,9572. Meskipun demikian, nilai presisi dan *F1-Score* yang dihasilkan masih lebih rendah dibandingkan metode *Canny*. Hasil ini menunjukkan bahwa kombinasi *Otsu* dan *Canny* mampu meningkatkan efisiensi waktu pemrosesan tanpa penurunan kinerja yang terlalu signifikan.

Secara keseluruhan, berdasarkan hasil yang ditunjukkan pada Tabel 3.2, metode *Canny* memberikan hasil terbaik dari sisi *accuracy*, *precision*, *recall*, dan *F1-Score*, namun membutuhkan waktu komputasi paling lama. Metode *Otsu* menghasilkan kinerja yang lebih rendah, sedangkan kombinasi *Otsu* dan *Canny* seimbang antara kinerja dan efisiensi waktu. Seluruh pengujian pada penelitian ini dilakukan dengan jumlah *epoch* sebanyak 30 sehingga perbandingan kinerja antar metode dilakukan pada kondisi pengujian yang sama dan hasil yang diperoleh dapat dibandingkan secara adil.

REFERENSI

[1] Kausar, S., & Nandyal, S. (2025). A focused study on Otsu's thresholding for segmenting images of paralysis-affected individuals. *International Journal for Research in Applied Science & Engineering Technology, 13(6).*

[2] Oliveira, R. De, Takeuchi, A., Ulbricht, L., Gustavo, F., Magrin, S., Itamarati, F., Ganacim, S., Fernandes, L. G., Eduardo, F., & Junior, J. U. (2022). Comparison of Traditional Image Segmentation Methods Applied to Thermograms of Power Substation Equipment.

[3] G, O. E., C, A. D., & N, O. J. (2025). Exploring the Effectiveness of Sobel , Canny , and Prewitt Edge Detection Algorithms on Digital Images.

[4] Sekehravani, E. A., Babulak, E., & Masoodi, M. (2020). Implementing canny edge detection algorithm for noisy image. *9(4)*, 1404–1410. https://doi.org/10.11591/eei.v9i4.1837

[5] Fuentes, A., Juan, A., Luna, G., & Carnicer, R. M. (2022). ORIGINAL RESEARCH PAPER GUD‐Canny : a real‐time GPU‐based unsupervised and distributed Canny edge detector. *Journal of Real-Time Image Processing, 19(3), 591–605. https://doi.org/10.1007/s11554-022-01208-0*